Rebuilding One App in Two Seconds: Run-Time Plugin Decomposition of a 4,347-~~Package Go Binary~~

Zach Kelling

Hanzo AI

Sequel to HIP-0106 Revision 1

Rebuilding One App in Two Seconds: Run-Time Plugin Decomposition of a 4,347-Package Go Binary

Zach Kelling*
Hanzo Industries Inc (Techstars '17)
Los Angeles, California
`https://hanzo.ai`

Sequel to HIP-0106 Revision 1

Abstract

The Hanzo unified cloud binary (HIP-0106) collapsed a microservices control plane into one Go process, eliminating the service mesh and the container-per-service tax. It also produced a build with 4,347 reachable packages, and we report a consequence that the original design did not anticipate: *editing one line in one of 107 applications costs the same as editing all of them.* We measure a one-app rebuild at 20.23 s with a 5.6 GB peak resident set, against a 19.49 s no-op rebuild — the marginal cost of the edit itself is under a second, and everything else is the link, which is a function of total reachable packages and is therefore paid in full regardless of what changed. We describe a decomposition that keeps the single-artifact deployment while removing the shared link. An application builds as its own binary and is composed into a host at run time over a unix-domain socket carrying the ZAP binary protocol; the host embeds those binaries with `go:embed` and still ships as one file. The composition primitive is a single function type, `Service = func(*App) error`, which a linked-in application and a separately built one both inhabit, so where a service runs becomes a deployment decision rather than a code change. Measured on the same tree and machine, a one-app rebuild falls to 2.11 s (9.6×) with a 0.63 GB peak resident set (8.9×), and the host does not rebuild at all. We explain why Go's `-buildmode=plugin` cannot serve this role, give the three invariants that make repeated hot reloads flat in memory, and report a second and larger finding: a single import in the root package placed 1,154 packages — every Kubernetes, AWS, gRPC and container SDK in the tree — beneath all 107 applications, and inverting it cut that floor from 1,798 packages to 644.

1 The cost that was not modelled

HIP-0106 argued the unified binary on operational grounds: in-process calls instead of network round-trips, no sidecars, no per-call mutual TLS, one artifact per brand. Those hold. What the design did not model is that a statically linked Go binary has a *shared* link, and the link is the dominant term in the edit–build–test loop.

Table 1 gives the baseline, measured warm on a 20-core `aarch64` host with a 28 GB Go build cache, using `/usr/bin/time` for wall clock and peak resident set. Each figure is the whole `go build` invocation, which is what a developer actually waits for.

Two consequences follow. First, the loop does not get faster by making any individual application smaller, because no application is the cost. Second, the 5.6 GB peak is per concurrent build: on a machine with 121 GB of RAM, a handful of parallel builds is enough to exhaust it, and we observed exactly that — an out-of-memory event

`cmd/cloud`, 4,347 pkgs	wall	peak RSS
no-op rebuild	19.49 s	5.6 GB
pure relink	20.33 s	5.6 GB
rebuild one small app	20.23 s	5.6 GB

Table 1: The three are the same number. Compilation of the edited package is noise; the link is the cost.

traced to concurrent Go links against a 24 GB `tmpfs`.

It is worth being precise about what does *not* help. The repository already generates 87 per-application `main` packages, each of which serves exactly one application. They do not help: each still links every application and disables the rest at run time via an enable-list, so `go list -deps ./cmd/product` returns the same 4,347 packages as `./cmd/cloud`. A runtime enable-list is not a build boundary.

2 Why not `-buildmode=plugin`

Go ships a plugin mechanism, and it is the obvious candidate. It is unusable here for three independent reasons, each sufficient on its own.

It requires cgo. Building with `CGO_ENABLED=0` fails outright with *"-buildmode=plugin requires external (cgo) linking, but cgo is not enabled"*. Our images build `CGO_ENABLED=0` to `scratch`; adopting plugins means adopting a libc base image across the fleet.

It requires byte-identical builds of every shared dependency between host and plugin. With 1,923 modules in the graph, that is a coupling stronger than the static link it replaces — the host and plugin must be built together to be loadable together, which is precisely the property we are trying to remove.

It cannot unload. Go has no `dlclose`: a loaded plugin's code, globals and goroutines are unreclaimable for the lifetime of the host. A mechanism whose purpose is to replace a running application with a new build cannot be built on a primitive that can never release the old one.

A child process has none of these properties. It releases every byte when it exits, it needs no build-time agreement with its host, and it works under `CGO_ENABLED=0`.

3 One type for both

The composition primitive is a single function type:

```
type Service func(*App) error
```

A unit of functionality attaches its own routes, middleware and shutdown hooks. A constructor that takes dependencies and returns one of these is the same thing curried, so a composition root only ever sees `Service`. The existing mount signature in the unified binary, `func(app *zip.App, deps Deps) error`, *is* this type curried on `deps`, so the 107 applications required no reshaping.

The load path returns the same type, which is the whole point:

```
//go:embed bin/billing
var billingBin []byte

app.Add(
    billing.New(deps),
    zip.Load("/v1/billing", zip.Plugin{
        Name: "billing", Bin: billingBin
    }),
    zip.Load("/v1/ml", zip.Plugin{
        Name: "ml", Addr: mlAddr}),
)
```

The three arguments are, respectively, an application linked into this binary, one that ships as its own binary embedded in it, and one already running elsewhere. They have the same type. Moving a service between those lines is a deployment decision, not a code change — and critically, the single-artifact property that motivated HIP-0106 survives, because `go:embed` puts the plugin binaries inside the host binary.

Serving and delegating are symmetric, and read the transport off the address scheme rather than off a method name:

```
app.Listen("/run/hanzo/billing.sock")
   // in the plugin
app.Mount("/v1/billing", sock)
   // in the host
```

The scheme names the protocol; the address names where it is spoken. A path is a unix socket, a `host:port` is TCP, so one wire never needs two schemes. A plugin is an ordinary application: no SDK, no schema, no code generation. Its only plugin-specific line is reading the address its host assigned.

4 Reload without leaking

Replacing a running application with a new build is the operational payoff, and it is where a naive implementation leaks. Three invariants hold.

Routes register once. They are registered at load and resolve their target per request through an atomic pointer. Re-registering on each reload would grow the route table without bound — the leak that makes hot reload untenable in practice.

The replacement must be listening before any request moves to it. The new process is started and proven to accept on its socket while the old one keeps serving. A build that fails to start changes nothing and returns an error; a bad deploy cannot take the route down.

The old process is released completely. Killed, reaped exactly once (one `Wait` per process, its result delivered on a channel), its pooled connections closed, its private directory removed.

Unloading leaves the routes registered and answering 503 rather than removing them, for the same reason: the route table is never mutated after load, so repeated load/unload cycles are flat.

These are verified rather than asserted. The test suite builds two genuinely different binaries from identical source, distinguished by a linker-stamped version, and asserts *which* one answered across four consecutive swaps, a deliberately failed swap, and an unload/restore cycle. Race detector clean.

5 Measurements

Table 2 compares the same application under both regimes, on the same tree, same machine, same warm cache. The plugin is one real application from the unified binary, built as its own binary; the host is a binary that links the web framework and the transport and no application at all.

	pkgs	**rebuild**	**peak RSS**
host only	313	0.65 s	0.31 GB
one app (plugin)	646	2.11 s	0.63 GB
monolith	4,347	20.23 s	5.6 GB

Table 2: Rebuilding one application. The monolith figure is the cost of editing *any* application; the plugin figure is the cost of editing *that* application, and the host does not rebuild at all.

The edit–rebuild loop improves 9.6× in wall clock and 8.9× in peak memory. Binary size falls from 497.1 MB to 43.3 MB for the plugin and 25.4 MB for the host.

The memory result matters more than the ratio suggests. At 5.6 GB per link, concurrency is bounded at roughly twenty parallel builds on a 121 GB machine before thrashing, and in practice far fewer alongside anything else. At 0.63 GB, the same machine sustains an order of magnitude more — and plugin builds are mutually independent, so they genuinely parallelise, whereas the monolith's link is one serial operation no amount of hardware shortens.

Link time scales with reachable packages rather than with edited code, which the three rows make legible: 313 packages link in 0.65 s, 646 in 2.11 s, 4,347 in 20.23 s.

6 The dependency floor

Decomposition exposed a larger and more embarrassing finding. Every application imports the root package to obtain its dependency struct, so the root package's import graph is the floor beneath all 107 of them. That floor was 1,798 packages, and a single import accounted for 1,154 of them.

The import was used for exactly four function-pointer registrations — a tier reader, a balance reader, a usage recorder, an ingest dialer — all pointing *outward*. Nothing read a value back. A second import, a Kubernetes client used by one

file to poll the live pod set, accounted for 263 more.

Both inverted trivially once observed. The root package declares the callbacks as plain function types and keeps the closures it builds; the composition root, which already links the heavy dependencies, performs the registration. The pod lister moved to its own package and is installed into an exported variable.

root package	before	after
total packages	1,798	644
`k8s.io/`	263	0
`aws/`	96	0
`grpc`	65	0
`go-git`	56	0
`cloud.google.com/`	18	0

Table 3: Inverting two outward-only imports. Behaviour unchanged.

The general lesson is worth stating plainly, because it is cheap to check and expensive to miss: *an import that only ever pushes values outward does not need to be an import.* Declare the type locally, let the composition root do the registration, and the dependency leaves the graph of everything downstream.

7 Limits

We report what these measurements do not establish.

The plugin figure of 646 packages still includes the 644-package root, because the application takes the shared dependency struct. A plugin that took only what it needs would be smaller again; we have not built one, and doing so requires narrowing the dependency struct per application.

Eighty-three direct import edges exist *between* application packages. Each is a compile-time symbol reference that must become an interface, an RPC call, or a merge before those two applications can occupy different processes. Seven packages are hubs. This is the real remaining work and it is not mechanical.

Several process-global couplings bind at load time rather than through the dependency struct — an indexer installed by one application into another, a dispatcher assembled at the composition root to break an import cycle, a single-process ledger resolved by every consumer of money. Each would break silently, not loudly, across a process boundary.

The per-organisation database handle cache assumes one process per `{service}.db` and is fenced by a single-writer lease. Applications that share a service name cannot split without splitting that ownership first.

Finally, the numbers are warm-cache, single-machine, single-run. They measure the developer loop, which is what we set out to improve; they are not cold-build or CI-throughput measurements, and the ratios should not be quoted as such.

8 Conclusion

The unified binary was the right call and remains so: one artifact, no mesh, no sidecars, in-process calls between subsystems. What it accidentally also bought was a shared link whose cost every developer pays on every edit, and a dependency floor that one careless import raised for 107 applications at once.

Neither required abandoning the design. Making the composition primitive a single function type meant a linked-in application and a separately built one became the same thing, so the boundary could be moved per deployment without touching code. Embedding the plugin binaries preserved the single artifact. Inverting two outward-only imports cut the shared floor by 64%.

The edit–rebuild loop went from 20.23 s and 5.6 GB to 2.11 s and 0.63 GB. The interesting part is not the ratio; it is that the cost was never in the code anyone was editing.